

Protocolo de Pruebas, Resultados y Validación

Centinela UV · TEI201 · Universidad Adolfo Ibáñez
Leo Castro · Sara Orellana · Cristian Yañez

1. Objetivo general

Comprobar que Centinela UV funciona como un sistema IoT completo, es decir, que:

1. mide radiación ultravioleta real y no entrega valores al azar,
2. clasifica correctamente esa medición en un nivel de riesgo,
3. comunica el nivel con el LED RGB,
4. envía las lecturas al servidor y estas quedan guardadas,
5. avisa por Telegram cuando el índice sube de nivel,
6. muestra la información de forma interpretable en el dashboard.

2. Equipo y condiciones

Todas las pruebas se hicieron con el prototipo completo: ESP32-S3 DevKitC-1, sensor UV GY-ML8511, LED RGB de cátodo común con resistencias de 220 Ω , batería LiPo 3.7 V y Li-Po Rider Pro, hotspot WiFi de un celular, servidor Hostinger (`guardar.php`, `test.php`, `dashboard`) y el bot de Telegram "Centinela UV Bot".

Umbral usado en todas las pruebas:

Índice UV	Nivel	LED	Mensaje de Telegram
0.0 – 2.9	BAJO	Verde	—
3.0 – 5.9	MODERADO	Amarillo	Aviso
6.0 o más	ALTO	Rojo	Alerta

Aclaración sobre las simulaciones. No siempre tuvimos sol disponible a la hora de probar. Por eso distinguimos dos tipos de prueba y lo señalamos en cada resultado:

- Prueba real: el sensor midiendo radiación solar al aire libre.
- Simulación: el sensor expuesto a una fuente de luz artificial cercana, o bien el **modo demostración**, en el que `test.php` fuerza el estado ALTO durante 8 segundos sin medir nada. Los mensajes que empiezan con `PRUEBA:` corresponden a este modo.

3. Pruebas

Prueba 1 — Lectura del sensor (real)

Objetivo: comprobar que el sensor mide radiación ultravioleta de verdad: que en condiciones de poca luz el valor se mantenga cerca de cero, y que suba cuando recibe más luz.

Condiciones: 23 de junio de 2026, ~18:05 hrs, en un balcón, al atardecer. Radiación esperada baja. ESP32 conectado por USB a un computador para registrar las lecturas.

Procedimiento:

1. Conectar el circuito completo (sensor, LED, ESP32).
2. Conectar el ESP32 al computador por USB.
3. Cargar el firmware y abrir el Monitor Serial.
4. Dejar el sensor quieto en el balcón y registrar las lecturas durante poco más de dos minutos.
5. Al final, acercar el sensor a una fuente de más luz para ver si el valor reacciona.

Criterio de éxito: en una hora de poca radiación el valor se mantiene cercano a cero, y al recibir más luz sube de forma clara y rápida.

Prueba 2 — Umbral y color del LED (simulación)

Objetivo: verificar que los tres niveles se activan en los cortes correctos y que el LED muestra el color que corresponde.

Procedimiento:

1. Con el sensor en sombra, verificar índice bajo 3.0 y LED **verde**.
2. Acercar gradualmente una fuente de luz hasta que el índice quede entre 3.0 y 5.9; verificar LED **amarillo**.
3. Acercar más la fuente hasta superar 6.0; verificar LED **rojo**.
4. Alejar la fuente y comprobar que el LED vuelve a amarillo y luego a verde.
5. Contrastar el color observado con el valor impreso en el Monitor Serial.

Criterio de éxito: el color del LED coincide siempre con el nivel que imprime el Monitor Serial, y los cambios ocurren en 3.0 y 6.0.

Prueba 3 — Alertas de Telegram (real) y (simulación)

Objetivo: comprobar que el bot avisa cuando el índice sube de nivel, y que no repite la alerta en cada ciclo.

Procedimiento:

1. Encender el dispositivo y verificar que llegue el mensaje `Sistema Monitor UV encendido`.
2. Subir el índice hasta MODERADO y verificar que llegue un `Aviso`.
3. Subir hasta ALTO y verificar que llegue una `ALERTA`.
4. Mantener el índice en ALTO durante un minuto y confirmar que **no** llegan mensajes repetidos.
5. Bajar el índice a BAJO y volver a subirlo; confirmar que llega una alerta nueva.

Criterio de éxito: llega un mensaje por cada cambio de nivel, con el índice correcto, y el chat no se llena de mensajes repetidos mientras el nivel se mantiene.

Prueba 4 — Envío y persistencia de los datos

Objetivo: comprobar que cada lectura llega al servidor y que sigue ahí después de apagar el dispositivo.

Procedimiento:

1. Encender el dispositivo con WiFi disponible.
2. Verificar en el Monitor Serial que la respuesta del servidor sea `Dashboard HTTP: 200`.
3. Dejarlo tomando datos durante unos minutos.
4. Apagar el ESP32.
5. Recargar el dashboard desde otro dispositivo y comprobar que las lecturas anteriores siguen visibles.

Criterio de éxito: el servidor responde 200 en cada envío y los datos permanecen disponibles después de apagar y reiniciar el ESP32.

Prueba 5 — Modo demostración

Objetivo: verificar que podemos disparar una alerta de UV ALTO a voluntad, para la presentación.

Procedimiento:

1. Activar `test.php` en el servidor.
2. Esperar hasta 6 segundos (intervalo de consulta del ESP32).
3. Verificar que llegue el mensaje `PRUEBA: alerta de UV ALTO`.
4. Verificar que el LED se ponga rojo durante 8 segundos y luego vuelva al color que corresponde a la medición real.

Criterio de éxito: la alerta de prueba se dispara de forma confiable y el sistema vuelve solo a su estado real. Este modo **no mide radiación:** fuerza el estado.

Prueba 6 — Dashboard

Objetivo: verificar que el dashboard convierte los datos en información interpretable.

Procedimiento:

1. Abrir el dashboard con el dispositivo encendido.

2. Comprobar que el valor mostrado coincide con el del Monitor Serial.
3. Comprobar que el borde de color y la recomendación cambian con el nivel.
4. Revisar que el gráfico del día y los valores máximo, mínimo y promedio se actualicen.

Criterio de éxito: el dashboard muestra el valor actual, una recomendación coherente con ese valor y un resumen del día que permite ver la evolución de la radiación.

4. Resultados — Prueba 1 (medición real)

Registramos 57 lecturas en poco más de dos minutos (18:05:49 – 18:07:52), el 23 de junio de 2026, en un balcón al atardecer.

La prueba tiene dos fases: las primeras 55 lecturas con el sensor quieto midiendo la luz del ambiente, y las 2 últimas después de acercarlo a una fuente de más luz. Los separamos porque mezclarlos distorsiona el promedio.

Métrica	Fase ambiente (55 lecturas)	Serie completa (57)
Promedio	0,34	0,44
Máximo	1,2 (18:06:54)	3,9 (18:07:52)
Mínimo	0,0	0,0
Mediana	0,3	0,3
Nivel	BAJO en las 55	BAJO salvo la última

Durante la fase ambiente los valores se mantuvieron muy bajos, todos dentro del nivel BAJO, que es exactamente lo que se espera a esa hora. El valor 0,0 aparece 12 veces, siempre de forma aislada entre lecturas positivas.

En los últimos 4 segundos, al exponer el sensor a más luz, el valor subió de forma clara:

```
18:07:48 → 0,3    (ambiente)
18:07:50 → 2,3    (expuesto)
18:07:52 → 3,9    (expuesto, nivel MODERADO)
```

Un salto de más de diez veces el promedio ambiental en dos lecturas consecutivas.

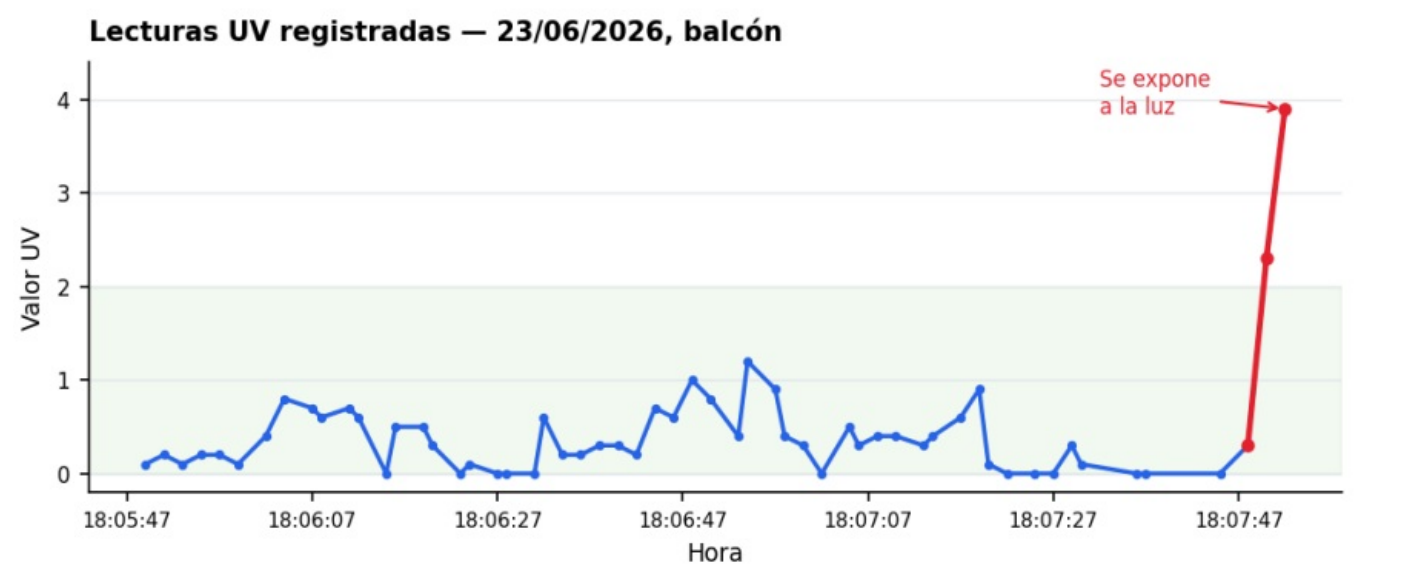


Figura 1. Las 57 lecturas registradas. Se observa una banda de ruido baja durante dos minutos y, al final, la subida vertical al exponer el sensor a la luz.

Conclusión: el sensor responde a la radiación real y no entrega datos sin sentido. La prueba cumple el criterio de éxito.

Sobre el registro: las lecturas se guardaron cada 2 o 3 segundos, pero hay tres huecos más largos (18:07:30 → 18:07:36 → 18:07:45). Corresponden a lecturas que no alcanzaron a registrarse mientras el ESP32 transmitía por WiFi. No afecta la conclusión, pero muestra que el registro no es de intervalo constante.

Los datos crudos están en `testing/datos/lecturas_23-06-2026.csv`, con una columna `fase` que distingue las lecturas de ambiente de las de exposición.

5. Resultados — Pruebas 2 y 3 (umbrales, LED y Telegram)

Estas pruebas se hicieron combinando medición real (con sol, durante la tarde) y simulación (con luz artificial cercana al sensor, y con el modo demostración), porque no siempre teníamos sol disponible a la hora de probar.

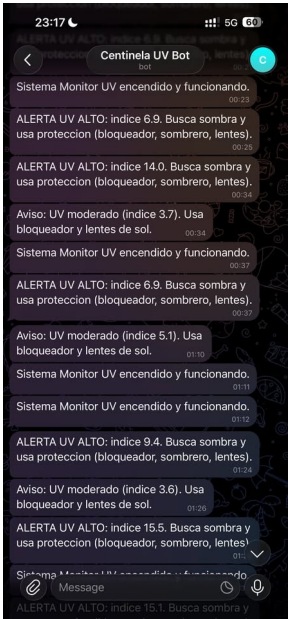


Figura 2. Alertas registradas durante las pruebas de simulación nocturna. Los índices altos (6.9, 14.0, 15.5) no corresponden a radiación solar: el sensor estaba expuesto a una fuente de luz artificial cercana. Los mensajes repetidos de "Sistema Monitor UV encendido" corresponden a reinicios del ESP32 (ver análisis de fallas).

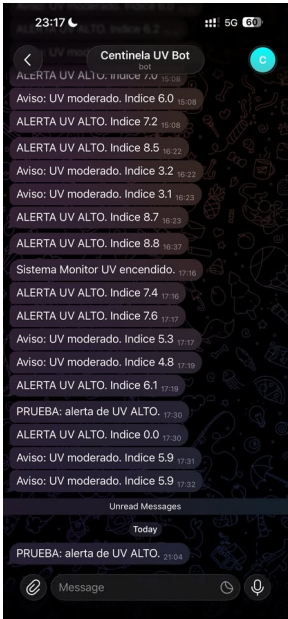


Figura 3. Alertas de las pruebas de la tarde, con radiación solar real (índices entre 3.1 y 8.8). A las 17:30 se ve el modo demostración: el mensaje PRUEBA : alerta de UV ALTO. seguido de ALERTA UV ALTO. Indice 0.0, porque el modo fuerza el estado sin medir.

Mensajes representativos:

Hora	Mensaje del bot	Tipo de prueba
15:08	ALERTA UV ALTO. Indice 7.0	Real (tarde con sol)
15:08	ALERTA UV ALTO. Indice 7.2	Real
16:22	Aviso: UV moderado. Indice 3.2	Real

16:37	ALERTA UV ALTO. Indice 8.8	Real
17:19	Aviso: UV moderado. Indice 4.8	Real
17:30	PRUEBA: alerta de UV ALTO.	Modo demostración
17:30	ALERTA UV ALTO. Indice 0.0	Modo demostración (estado forzado)
00:25	ALERTA UV ALTO: indice 6.9	Simulación con luz artificial
00:34	ALERTA UV ALTO: indice 14.0	Simulación con luz artificial
01:24	ALERTA UV ALTO: indice 9.4	Simulación con luz artificial
21:04	PRUEBA: alerta de UV ALTO.	Modo demostración

Lo que muestran estos resultados:

- El LED y el mensaje siempre coincidieron con el nivel calculado: verde bajo 3.0, amarillo entre 3.0 y 5.9, rojo sobre 6.0.
- El sistema **no repite** la alerta mientras el nivel se mantiene: solo avisa cuando el nivel cambia. En las capturas se ve que entre dos alertas de UV ALTO siempre hubo un descenso previo de nivel.
- Los índices altos registrados de madrugada no son radiación solar: son simulaciones con una fuente de luz artificial. Las anotamos como tales para no confundir simulación con medición.
- Los mensajes que empiezan con PRUEBA: corresponden al modo demostración, en el que el sistema fuerza el estado ALTO durante 8 segundos sin medir.

6. Resultados — Pruebas 4 y 6 (persistencia y dashboard)

El Monitor Serial mostró `Dashboard HTTP: 200` en cada envío, lo que confirma que el servidor recibió y guardó la lectura. Al apagar el ESP32 y volver a abrir el dashboard desde otro dispositivo, las lecturas anteriores seguían disponibles: **los datos persisten más allá de la sesión**, que es justamente lo que distingue un sistema IoT de un simple monitor serial.

El dashboard mostró en todo momento el mismo valor que el Monitor Serial, con la recomendación de protección correspondiente al nivel y el gráfico del día con máximo, mínimo y promedio.

7. Análisis de fallas encontradas

Falla	Causa	Solución aplicada
Las lecturas saltaban de un segundo a otro (por ejemplo entre 0 y 1,2) aunque las condiciones no cambiaran	Ruido normal del ADC y del sensor, sobre todo con poca luz	El firmware toma 5 lecturas seguidas y las promedia antes de mostrar el valor. Si quisiéramos afinarlo más, podríamos aumentar la cantidad de muestras
Un mensaje decía " Aviso: UV moderado. Índice 6.0 ", aunque 6.0 debería ser nivel ALTO	El valor real era 5,95: la clasificación usa el número completo, pero el mensaje lo imprime redondeado a un decimal	Es un problema de presentación, no de lógica. La solución sería mostrar dos decimales o clasificar sobre el valor ya redondeado
Se repitió varias veces el mensaje " Sistema Monitor UV encendido " en pocos minutos	El ESP32 se reiniciaba. Lo atribuimos a caídas de tensión cuando el módulo WiFi transmite y la batería está baja	Probamos alimentando desde el Li-Po Rider Pro con la batería cargada, y los reinicios desaparecieron
El modo demostración envía " ALERTA UV ALTO. Índice 0.0 "	La función <code>revisarPrueba()</code> fuerza el nivel a ALTO pero el índice sigue siendo el medido	No es un error de medición, es el comportamiento esperado del modo demo. Lo declaramos explícitamente para no confundirlo con una medición real

8. Validación contra el problema original

En el Avance 1 identificamos que **las personas que pasan tiempo al aire libre no tienen una forma directa de saber cuándo la radiación UV es alta y deben protegerse**.

Las pruebas muestran que Centinela UV entrega exactamente esa información:

1. **Captura.** La Prueba 1 demuestra que el sensor mide radiación real: se mantiene cerca de cero cuando hay poca luz y sube cuando aumenta.
2. **Información.** El sistema no muestra un voltaje crudo: lo convierte en un índice UV y lo clasifica en tres niveles asociados a una recomendación concreta de protección.
3. **Decisión.** El usuario recibe esa recomendación por tres vías con distinto alcance: el LED (para quien está al lado del dispositivo), el dashboard (para revisar el día) y la alerta de Telegram (para enterarse aunque no esté mirando nada).

La evidencia de Telegram es la más directa: durante las pruebas de la tarde, el bot avisó automáticamente cuando el índice cruzó 3.0 y cuando superó 6.0. Ese aviso, llegando al celular sin que el usuario haga nada, es justo la información que faltaba en el problema que planteamos.

El ciclo IoT queda cerrado:

El dispositivo captura datos → genera información → permite tomar decisiones

9. Conclusión y mejoras futuras

Centinela UV cumple el objetivo del proyecto: mide radiación ultravioleta real, la guarda, la muestra y avisa cuando hay riesgo.

Las limitaciones que reconocemos, y que serían el siguiente paso:

- **Calibrar el sensor** contra un medidor UV de referencia. Hoy usamos la relación aproximada de la hoja de datos del ML8511, así que el valor absoluto puede tener error aunque la tendencia sea correcta.
- **Validar con sol de mediodía**, para confirmar los umbrales altos con radiación real y no solo con luz artificial.
- **Guardar las lecturas localmente** cuando no hay WiFi, y enviarlas al recuperar la conexión.
- **Limpiar los campos `temp` y `hum`** que el firmware envía siempre en 0 y que quedaron de una versión anterior del proyecto.
- **Medir el nivel de batería** para avisar cuando quede poca carga.